# SpikeYOLO for Fire and Smoke Detection: Direct, Poisson and Latency Spike Inputs

Code of the paper **"Spiking YOLO for Fire and Smoke Detection in Ground and UAV Images: A Comparison of Direct, Poisson, and Latency Spike Inputs"** (P. Bujons, R. Parada; submitted to *Fire*, MDPI).

The study is a **controlled comparison of three ways of presenting an RGB image to the same spiking detector**
(SpikeYOLO), with the detector, the pretrained initialization, the data and the training recipe fixed:

| input | what the first layer delivers to the network (`[T, B, 3, 640, 640]`) |
|---|---|
| **direct RGB** | the image repeated at each of the `T` timesteps (original SpikeYOLO) |
| **Poisson** | at each timestep every pixel emits a Bernoulli spike with probability equal to its intensity |
| **latency** | one spike per pixel at timestep `floor((1 - x) * T)` (bright pixels first; `x = 0` never spikes) |

`T = 4` and `T = 6` are evaluated. **YOLOv8m** and a lightweight **VanillaCNN** (ResNet18 backbone, one box per grid cell) are trained and evaluated with the same protocol as references. No new architecture is proposed: the three SpikeYOLO configurations have exactly the same layers and parameters (23.1 M); only the content of the input tensor changes.

The repository contains the training scripts, the unified evaluation (accuracy with bootstrap intervals), the efficiency benchmark (latency, GPU energy, CO2), the near-duplicate leakage analysis, and the video analysis.

> **What the results say (and do not say).** On D-Fire, YOLOv8m obtained the best detection performance (test mAP@0.5 0.753), followed by direct-RGB SpikeYOLO (0.701, `T=4`), Poisson (0.624) and latency (0.590). On a conventional GPU, the SpikeYOLO configurations needed 5 to 8 times more inference time and 7 to 10 times more GPU energy per image than YOLOv8m, independently of the input representation. **The paper does not show an energy advantage of spiking detectors on a GPU**; the networks are executed as dense tensors, and neither event cameras nor neuromorphic hardware were used. This repository is the GPU reference point and a first step toward evaluating the same detectors on neuromorphic hardware. See the paper for all numbers and limitations (single training run per configuration, `T in {4, 6}`, near-duplicates between D-Fire splits, shared GPU).

## 1. Repository layout

The scripts assume this folder structure (`code_paper_2/` is the project root; the names matter because the evaluation code looks for them):

```
code_paper_2/
├── SpikeYOLO/            fork of SpikeYOLO (Ultralytics 8.0.197) with the
patched input layer (see section 2.2)
│   ├── train_spikeyolo.py        training of the three input configurations (-
-encoding rgb|poisson|latency)
│   └── 23M_best.pt               pretrained SpikeYOLO checkpoint (from the
SpikeYOLO README; not included)
├── ultralytics/          Ultralytics 8.3.231 (YOLOv8m)
│   └── train_yolov8.py
├── VanillaCNN/           train_vanillacnn.py, model.py, loss.py
├── datasets/
│   ├── clean_labels.py           removes degenerate label lines (width or
height <= 0)
│   ├── dataset_report.py         audit of the splits (counts, sizes,
categories, duplicates)
│   └── D-Fire_clean/             train/ val/ test/ (images/ + labels/) and
data.yaml
├── queue/                jobs.py + run_queue.py: the whole set of training
runs, unattended and resumable
├── eval/                 unified evaluation, benchmark, leakage, qualitative
and video tools
├── Results_train/        one folder per trained model (weights/best.pt,
epoch_metrics.csv, ...)
└── Results_test/         predictions, metrics, benchmark and video outputs
(written by eval/)
```

Models are named `spikeyolo_{rgb|poisson|latency}_T{4|6}_s{seed}`, `yolov8m_s{seed}`, `vanillacnn_s{seed}` (25 epochs) and `vanillacnn_long_s{seed}` (180 epochs). The evaluation discovers every finished run (`Results_train/<name>/weights/best.pt`) from these names.

# 2. Installation

## 2.1 Environment used in the paper

| | |
|---|---|
| GPU | NVIDIA GeForce RTX 5090 (32 GB), driver 580.105.08 |
| CPU / OS | AMD Ryzen 9 9950X3D, openSUSE Leap 15.6 |
| Python / PyTorch | 3.11 / 2.10 (CUDA 12.8) |
| Ultralytics | 8.0.197 (SpikeYOLO fork and VanillaCNN data pipeline), 8.3.231 (YOLOv8m) |
| CodeCarbon | 3.2.3 (emissions only) |
| NVML | `nvidia-ml-py` (GPU energy counter) |

Two different Ultralytics versions are needed, so **each model family runs in its own process** (the evaluation does this for you; never import both in one interpreter).

```
python -m venv tfm_paper && source tfm_paper/bin/activate
pip install torch torchvision            # a build that matches your CUDA
pip install numpy scipy pandas matplotlib opencv-python pyyaml nvidia-ml-py
codecarbon==3.2.3
# SpikeYOLO fork: install its requirements from its own README
```

### 2.2 The input layer (`MS_GetT`)

Layer 0 of SpikeYOLO (`MS_GetT` in `ultralytics/nn/modules/yolo_spikformer.py`) was replaced by a version with three modes, selected with `--encoding` at training time:

- `repeat` (direct RGB, the original behaviour; results with this mode are identical to the unmodified network),
- `poisson` (Bernoulli spikes; in evaluation the generator is seeded, so validation and test are reproducible),
- `latency` (`floor((1 - x) * T)`).

The encoding is computed **on the GPU, as the first layer of the model**, on the augmented image of every training iteration and at inference time; no encoded dataset is stored. The patch is in `MS_GetT_patch.txt`; apply it to the fork before training.

### 2.3 Data and checkpoints

1. Download the D-Fire dataset and place it in `datasets/D-Fire/`.
2. The paper keeps only images whose width and height are both at least 420 pixels (7,349 of the 21,527 images are removed). Apply the size filter, then remove degenerate labels:

   ```
   cd datasets
   python clean_labels.py --root D-Fire --out D-Fire_clean        # add --
   dry-run to only list what would change
   python dataset_report.py ...                                    # audit
   of the final splits (see its --help)
   ```

   The result is 14,178 images: 10,267 train, 1,125 validation, 2,786 test, with `D-Fire_clean/data.yaml` (add `test: test/` to be explicit).
3. Put the SpikeYOLO 23M pretrained checkpoint in `SpikeYOLO/` (file name set in `queue/jobs.py`, `SPIKE_WEIGHTS`) and `yolov8m.pt` in `ultralytics/`.

## 3. Training

### 3.1 The whole study: the queue

`queue/jobs.py` defines every run with one common recipe (25 epochs; SGD, lr 0.01 decayed to `0.01 * 0.004`, momentum 0.937, weight decay 5e-4, 3 warm-up epochs, effective batch 64 by gradient accumulation, mosaic switched off in the last epochs, mixed precision, seed 0, per-step batch 4 for `T=4` and 2 for `T=6`; VanillaCNN with Adam 1e-4 and cosine schedule, for 25 and 180 epochs).

```
cd queue
python run_queue.py --dry-run                 # print what would run
nohup python run_queue.py > queue.out 2>&1 &   # or inside tmux
python run_queue.py --retry-failed             # run again what failed
python run_queue.py --only spikeyolo_rgb_T4_s0 # a single job
touch STOP                                     # stop after the current job
QUICK=1 python run_queue.py --state queue_state_quick.json   # smoke test: 2
epochs on 5% of the images
```

The queue is resumable: finished jobs are skipped and an interrupted job is resumed from `last.pt`. Logs go to `queue/logs/<name>.log`.

### 3.2 A single run

```
cd SpikeYOLO
python train_spikeyolo.py --data ../datasets/D-Fire_clean/data.yaml --model
snn_yolov8s.yaml \
    --weights 23M_best.pt --T 4 --encoding poisson --epochs 25 --batch 4 --nbs
64 --amp \
    --name spikeyolo_poisson_T4_s0 --project ../Results_train

cd ../ultralytics   && python train_yolov8.py    --data ../datasets/D-
Fire_clean/data.yaml --name yolov8m_s0
cd ../VanillaCNN    && python train_vanillacnn.py --data ../datasets/D-
Fire_clean/data.yaml --epochs 180 --name vanillacnn_long_s0
```

`--encoding` accepts `rgb`, `poisson` and `latency`. Use `--fraction 0.05 --epochs 2` for a quick test.

## 4. Evaluation

Everything is under `eval/` and follows one protocol for all models: square 640x640 letterbox, NMS (conf 0.001, IoU 0.7, at most 300 detections), one script that produces the predictions and one numpy implementation of the metrics. AP/mAP use all the detections after NMS; the per-model confidence threshold (chosen on the validation set by maximizing F1) is used **only** for precision, recall, F1 and TP/FP/FN.

```
cd eval
python selftest_metrics.py && python selftest_pipeline.py          # synthetic
tests, no GPU

# check the pipeline on one finished model against its training log
python dump_predictions.py --model yolov8m_s0 --split val

# accuracy of every finished run (5 Poisson generator seeds; deterministic
models run once)
python run_eval.py --stage accuracy --repeats 5
```

```
# time, GPU energy and CO2: nothing else may run on the GPU (the script refuses
if it is busy)
nohup setsid python run_eval.py --stage bench --repeats 5 --fuse no >
eval_bench.log 2>&1 &

# tables (accuracy with bootstrap intervals, paired differences, efficiency
with intervals and tests)
python run_eval.py --stage aggregate --carbon-intensity <g CO2/kWh>
```

Outputs are written to `Results_test/`. The files of `eval/`:

| file | purpose |
|---|---|
| `models.py` | finds the finished runs, loads each model family in its own process, builds the data loader |
| `dump_predictions.py` | every prediction of one model on one split, saved as `.npz` |
| `metrics.py` | mAP@0.5, mAP@0.5:0.95, per-class AP, validation threshold, TP/FP/FN, recall by target size, bootstrap CIs |
| `benchmark_efficiency.py` | latency and GPU energy (NVML counter minus idle): network only (encoding bypassed), encoding only, end to end; CodeCarbon for CO2 |
| `aggregate.py` | accuracy and efficiency tables, paired bootstrap differences, Welch and Mann-Whitney tests with Holm correction |
| `run_eval.py` | runs the stages above, resumable, models interleaved across benchmark repeats |
| `check_leakage.py` | near-duplicate images between splits (difference hash + thumbnail correlation) |
| `eval_subset.py` | re-evaluates the models on the test images without near-duplicates (no retraining) |
| `make_qualitative.py` | qualitative figure of detections on random test images |
| `predict_video.py`, `make_video_figures.py` | video analysis (section 6) |

Efficiency protocol: batch size 1, single precision, no layer fusion (so every model follows the same code path), five repetitions in a different random order, GPU energy read from the NVML cumulative counter with the idle power subtracted.

### 4.1 Near-duplicate analysis

```
python check_leakage.py                    # loose criterion: Hamming <= 8,
NCC >= 0.95 (see --max-hamming, --min-ncc)
python eval_subset.py                      # metrics on the test images
```

```
without a near-duplicate
python make_qualitative.py --exclude
../Results_test/leakage/leakage_test_confirmed.txt --seed 0
```

In the paper, 62.2% (loose criterion) and 56.7% (strict criterion) of the test images have a near-duplicate in the training or validation splits (consecutive frames of fixed cameras, repeated photographs). Scores on D-Fire are therefore optimistic; the ranking of the models is preserved on the images without near-duplicates.

## 5. Reading the results correctly

- Intervals are bootstrap intervals over the test images. **Each configuration was trained once (seed 0)**, so the variability between training runs is not measured; the Poisson input is the only one evaluated with several (generator) seeds.
- `T = 6` runs used a per-step batch of 2 instead of 4, so the effect of `T` is confounded with batch-normalization statistics.
- The GPU is **shared with other users**. Absolute energy values depend on the session (a later re-run gave 30 to 39% more energy for the same latency); the comparison between models rests on their **ratios** (which changed by less than 7%). Do not compare absolute energies between the image benchmark and the video benchmark.
- GPU energy is measured with the NVML counter; CodeCarbon is only used for CO2 and is a software estimate.
- The networks run as dense tensors on a GPU: the sparsity of the spikes is not exploited. The results do not apply to neuromorphic hardware.

## 6. Video analysis (qualitative)

Videos are **not annotated**: no accuracy is computed. Each frame is processed independently with the same letterbox, network input and per-model threshold as the test-set evaluation. One model per process:

```
cd eval
python predict_video.py --model yolov8m_s0              --videos ../videos --
measure 300
python predict_video.py --model spikeyolo_rgb_T4_s0     --videos ../videos --
measure 300
python predict_video.py --model spikeyolo_poisson_T4_s0 --videos ../videos --
measure 300
python predict_video.py --model spikeyolo_latency_T4_s0 --videos ../videos --
measure 300

# optional, exploratory: the T latest frames fed to the T timesteps (direct and
Poisson only; not a trained mode)
python predict_video.py --model spikeyolo_rgb_T4_s0     --videos ../videos --
window
python predict_video.py --model spikeyolo_poisson_T4_s0 --videos ../videos --
window

# figures and tables (no GPU); --crop drops frames, e.g. an advertising end
```

```
card (end exclusive)
python make_video_figures.py --frames 4 --dpi 300 --crop big_fire1=0:1231
```

Outputs per video and model (`Results_test/videos/<video>/<model>/`): `annotated.mp4` (smoke in blue, fire in red), `detections.npz` and, with `--measure`, `timing.json` (ms and net GPU mJ per frame). `make_video_figures.py` writes the frames figure, a side-by-side video and `video_summary.csv` (share of frames with smoke / fire detections, boxes per frame, cost). The share of frames describes the response of each model; it is not accuracy, since there is no ground truth. The three videos used in the paper are public YouTube videos listed in the paper's references.

## 7. Reproducing the paper

| **paper** | **command / file** |
|---|---|
| Training protocol table, validation curves | `queue/`, `collect_and_plot.py` (reads `Results_train`) |
| Test accuracy with intervals, per-class results, recall by size | `run_eval.py --stage accuracy`, `--stage aggregate` |
| Near-duplicate analysis | `check_leakage.py`, `eval_subset.py` |
| Inference time, energy, CO2 | `run_eval.py --stage bench`, `--stage aggregate` |
| Qualitative images | `make_qualitative.py` |
| Video results and cost | `predict_video.py`, `make_video_figures.py` |

## 8. Citation and license

Manuscript in Preparation

This repository builds on SpikeYOLO and Ultralytics (AGPL-3.0) and on the D-Fire dataset. TODO: state the license of this repository (derivative code of the Ultralytics-based forks must keep their license) and the license terms of the three videos.